

Politechnika Rzeszowska
Wydział Matematyki i Fizyki Stosowanej

Sprawozdanie

Alorytmy i struktury danych

ZADANIE PROJEKTOWE/ALGORYTMICZNE

Autor:

MATEUSZ DZIAŁOWSKI

20 stycznia 2026

Spis treści

1	Wstęp	2
2	Opis problemu	3
3	Podstawy teoretyczne	3
3.1	Złożoność algorytmu	3
4	Szczegóły implementacji	3
4.1	Rozwiązanie problemu - podejście pierwsze	3
4.1.1	Analiza problemu	3
4.1.2	Schemat blokowy algorytmu	5
4.1.3	Algorytm zapisywany w pseudokodzie	6

1 Wstęp

Celem niniejszego opracowania projektu jest przedstawienie implementacji oraz analizy wydajnościowej algorytmu, rozwiązującego problem wyszukiwania specyficznych podciągów w zadanym ciągu liczb całkowitych dodatnich wejściowych. Dokumentacja obejmuje szczegółowy opis problemu, podstawy teoretyczne, implementację algorytmu oraz wyniki testów wydajnościowych przeprowadzonych na różnych zestawach danych wejściowych.

"W przedstawionym opracowaniu indexowanie pozycji w podciągach i tablicach zaczyna się od 0."

2 Opis problemu

Dla zadanego ciągu liczby całkowitych dodatnich ciągu $A[]$ o długości N , należy znaleźć wszystkie podciągi o długości $M = 5$ (pięcioelementowe) w danym ciągu, które spełniają kryterium: Suma elementów na pozycji 2 i 4 jest większa od sumy elementów na pozycji 1, 3 i 5.

Dla, podciągu $T[] = [t_0, t_1, t_2, t_3, t_4]$, warunek przyjmuje postać:

$$t_1 + t_3 > t_0 + t_2 + t_4 \quad (1)$$

Przykład: Dla ciągu $A[] = [1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1]$, zgodnie z powyższym kryterium, podciągi spełniające warunek to:

- $T[M] = [1, 130, 1, 9, 11]$, ponieważ $(130 + 9 > 1 + 1 + 11)$
- $T[M] = [1, 9, 11, 6, 1]$, ponieważ $(9 + 6 > 1 + 11 + 1)$
- $T[M] = [1, 1, 3, 1, 1]$, ponieważ $(1 + 1 > 1 + 3 + 1)$

Jak widać, w powyższym przykładzie istnieją trzy podciągi spełniające zadane kryterium.

3 Podstawy teoretyczne

Algorytmy opiera się na technice przesuwania okna o stałą $M = 5$. Dla każdego możliwego podciągu o długości M w ciągu $A[]$, algorytm sprawdza, czy spełnia on warunek określony w równaniu (1). Jeśli tak, podciąg jest zapisywany do wyniku.

3.1 Złożoność algorytmu

Teoretyczna analiza złożoności algorytmu przedstawia się następująco:

- **Złożoność czasowa:** Sprawdzenie jednego okna zajmuje czas stały $O(1)$. Ponieważ okno przesuwamy przez cały ciąg o długości N , wykonujemy $N - M + 1$ operacji, M jest stałą co powoduje że teoretyczna złożoność powinna wynosić $O(N)$.
- **Złożoność obliczeniowa:** Algorytm wymaga przechowywania ciągu wejściowego o wielkości N oraz tablicy wyników M , która jest stałą. Równanie wygląda następująco: $O(N + M)$, co teoretyczne w najgorszym przypadku, gdy każdy podciąg spełnia warunek złożoność pamięciowa wynosi $O(N)$.

4 Szczegóły implementacji

4.1 Rozwiązanie problemu - podejście pierwsze

4.1.1 Analiza problemu

Choć algorytm jest stosunkowo prosty, jego implementacja wymaga uwzględnienia kilku kluczowych aspektów, takich jak efektywne zarządzanie pamięcią, optymalne podejście przy wykorzystaniu pętli oraz optymalizacja operacji sprawdzania warunków, co może znacząco wpłynąć na wydajność końcowego rozwiązania. Pod uwagę trzeba wziąć również obsługę różnych przypadków brzegowych, takich jak bardzo krótkie ciągi wejściowe lub ciągi nie spełniające żadnego z kryteriów co wciąż wpływa na wydajność implementacji.

W podejściu pierwszym, postąpimy w sposób bezpośredni, iterując przez każdy możliwy podciąg o długości M w ciągu $A[N]$ i sprawdzając, czy spełnia on warunek określony w równaniu (1). Jeśli tak, dodajemy go do listy wyników, liście wyników nie przypisujemy żadnej wartości elementowej ponieważ przyjmujemy, że tablica dynamicznie zmienia swoje rozmiary. Również musimy wziąć pod uwagę przypadki, gdy długość ciągu N jest mniejsza niż M lub dane wejściowe są ułożone w sposób uniemożliwiający znalezienie podciągu spełniającego warunek. W takim przypadku nie ma możliwości znalezienia żadnego podciągu o wymaganej długości, więc algorytm powinien zwrócić pustą listę wyników.

1. Dane wejściowe.

Naszymi danymi wejściowymi są:

- Ciąg liczb całkowitych dodatnich $Tab[]$ również,
- długość ciągu N ,
- stała $M = 5$.

2. Dane wyjściowe.

Naszymi danymi wyjściowymi są:

- Tablica wyników $Wynik[]$, zawierająca wszystkie podciągi.

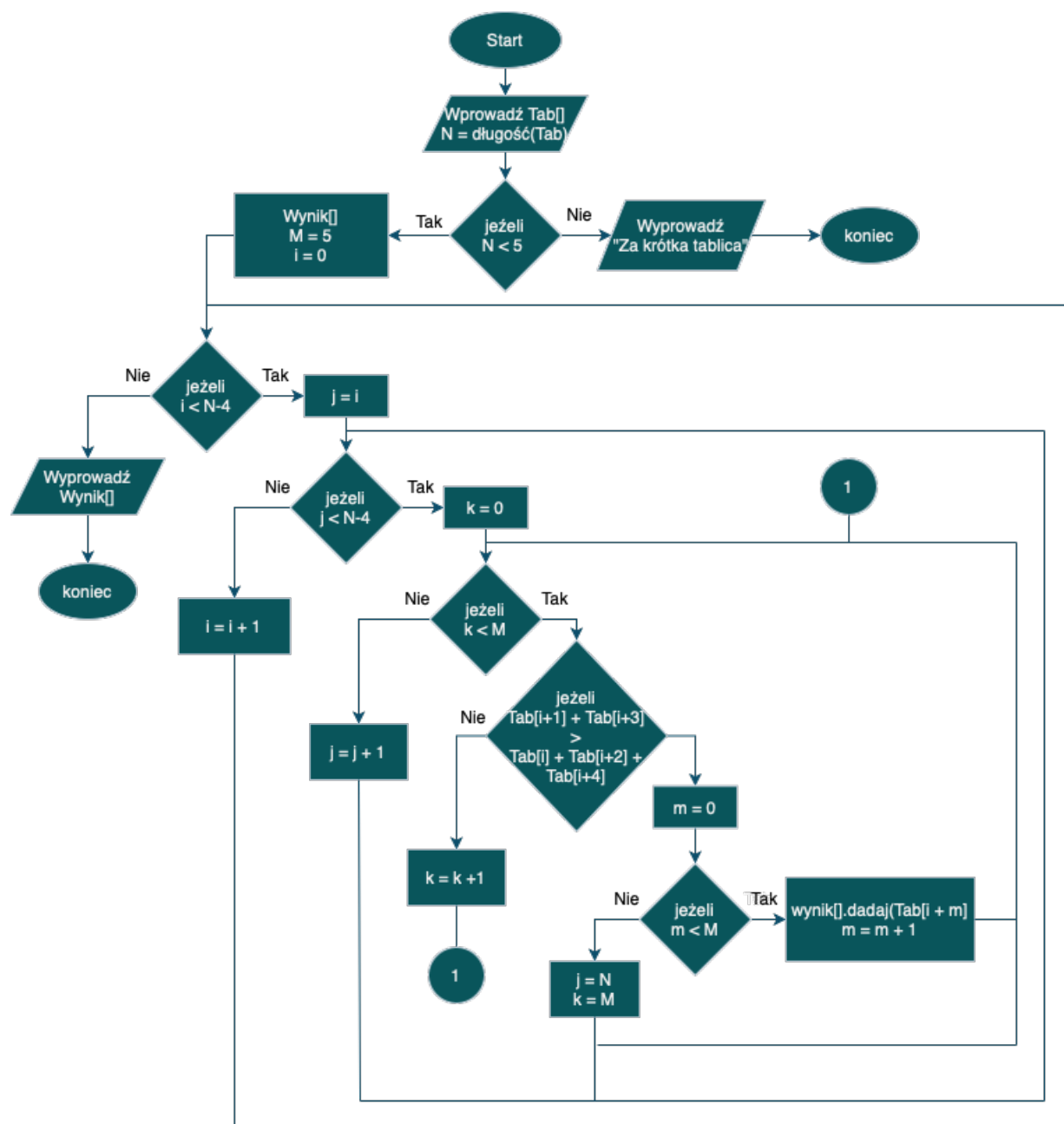
Implementacja wykorzystuje zagnieżdżone pętle iteracyjne (warunki zapętłone), wraz z zmiennymi sterującymi, aby efektywnie przeszukiwać ciąg wejściowy i identyfikować podciągi spełniające określone kryteria. Poniżej przedstawiono szczegółowy opis działania poszczególnych pętli:

- **Pętla zewnętrzna $i < N-4$ (i)** - przesuwa okno przez cały ciąg (od 0 do $N-4$), że względu na warunek sprawdzający (1) musimy pętle ograniczyć do $N-4$, w celu uniknięcia błędów. Przedstawionej pętli sprawdzamy krok po kroku każdy możliwy podciąg o długości $M = 5$, czy spełnia on warunek z równania (1), również pętla przesuwa okno o jeden element do przodu w każdej iteracji w celu sprawdzenia następnych możliwych podciągów.
- **Pętla wewnętrzna $j < N-4$ (j)** - rozpoczyna się od pozycji $j = i$ i iteruje do $N-4$, sprawdzając warunek z równania (1) dla każdego podciągu zaczynającego się na pozycji i .
- **Pętla iteracyjna $k < M$ (k)** - przechodzi przez elementy podciągu (od 0 do $M-1$), aby obliczyć sumy wymagane do sprawdzenia warunku z równania (1). W tej pętli sumujemy odpowiednie elementy podciągu, aby porównać je zgodnie z kryterium określonym w równaniu (1).
- **Pętla kopiująca $m < M$ (m)** - przepisuje spełniający warunek podciąg do tablicy wyników $Wynik[]$. Ta pętla iteruje przez elementy podciągu i kopiuje je do tablicy wyników, gdy warunek z równania (1) jest spełniony.

Gdy warunek z równania (1) zostanie spełniony dla danego okna, algorytm kopiuje 5 elementów do tablicy wyników i przerywa wewnętrzne pętle poprzez ustawienie wartości sterujących ($j = N$, $k = M$), co wymusza przejście do następnej pozycji okna.

4.1.2 Schemat blokowy algorytmu

Poniżej przedstawiono schemat blokowy ilustrujący działanie algorytmu:



Rys. 1: Schemat blokowy algorytmu wyszukiwania podciągów

4.1.3 Algorytm zapisywany w pseudokodzie

W poniższym pseudokodzie przedstawiono szczegółową implementację algorytmu. W przypadku pseudokodu, przyjęliśmy wykorzystanie pętli while do iteracji przez elementy ciągu oraz do sprawdzania warunków w porównaniu do schematu blokowego gdzie przyjęliśmy warunki jako pętle.

W pseudokodzie użyto następujących oznaczeń:

- $\text{Tab}[]$ - tablica wejściowa zawierająca ciąg liczb całkowitych dodatnich,
- N - długość tablicy wejściowej,
- $\text{wynik}[]$ - tablica wynikowa przechowująca znalezione podciągi,
- M - stała określająca długość podciągu ($M = 5$),
- i, j, k, m - zmienne sterujące pętlami.

```
Input  :  $\text{Tab}[], N$ 
Output:  $\text{wynik}[]$  lub  $-1$ 
if  $N < 5$  then
   $\text{return } -1$ 
 $\text{wynik} \leftarrow []$ ;
const  $M \leftarrow 5$ ;
 $i \leftarrow 0$ ;
while  $i < N - 4$  do
   $j \leftarrow i$ ;
  while  $j < N - 4$  do
     $k \leftarrow 0$ ;
    while  $k < M$  do
      if  $\text{Tab}[i + 1] + \text{Tab}[i + 3] > \text{Tab}[i] + \text{Tab}[i + 2] + \text{Tab}[i + 4]$  then
         $m \leftarrow 0$ ;
        while  $m < M$  do
           $\text{wynik.dodaj}(\text{Tab}[i + m]);$ 
           $m \leftarrow m + 1$ ;
         $j \leftarrow N$ ;
         $k \leftarrow M$ ;
      else
         $k \leftarrow k + 1$ ;
     $j \leftarrow j + 1$ ;
   $i \leftarrow i + 1$ ;
return  $\text{wynik}$ ;
```

Jak widać, algorytm iteruje przez każdy możliwy podciąg o długości M w ciągu $\text{Tab}[]$, sprawdzając, czy spełnia on warunek określony w równaniu (1). Jeśli tak, podciąg jest dodawany do tablicy wyników $\text{wynik}[]$. W przypadku, gdy długość ciągu N jest mniejsza niż M , algorytm zwraca -1 , wskazując na brak możliwości znalezienia podciągu spełniającego warunek.